

AMERICAN UNIVERSITY OF BEIRUT

MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE

Department of Electrical and Computer Engineering

EECE655 – Assignment 1

Zeinab Mazraani
ID: 202305614
zam25@mail.aub.edu

Tamer Slim
ID: 202400090
tws05@mail.aub.edu

EECE 655: Internet Security
Prof. Imad ElHajj
Fall 2026–2027

I. Introduction

An overlapping fragment attack is a network-layer attack that abuses IPv4 fragmentation and packet reassembly to bypass firewall rules. The IPv4 standard never specifies how to resolve two fragments that claim the same byte range, an ambiguity attackers have exploited for decades, from early evasion techniques to the more recent FragmentSmack denial-of-service vulnerability (CVE-2018-5391). A usecase of such an attack would be presenting an allowed TCP destination port to the firewall and later overwriting it with a blocked port, or to hide malicious payload data. This project builds an attacker and a matching detector around that ambiguity, escalating both across three versions.

II. Tool Descriptions

A. Attacker

The OverlapCraft Attacker tool is made up of a total of three versions, with each one building on the last.

General Mechanism All these versions share the same foundation layer, in which, each IP header built and sent is assembled "by hand" using raw sockets, rather than using dedicated Python library tools (e.g. scapy). This gives the attack a more granular control to craft the packets as deemed necessary. Some constants are fixed (e.g. TTL 64, IHL 5, UDP as the payload protocol, etc.), typically set to their default values (for Kali Linux OS, which is the attacker's OS), while others are random (e.g. Identification Field). In general, once a packet is ready to be sent, its header is built field by field, and includes the source and destination IP address (entered by attacker manually at each run), version (IPv4), TTL, IHL, service type, ID Field, flags, offset, protocol, and checksum (calculated using a 16-bit one's complement addition with end-around carry). Each fragment's payload, whether a repeated letter or randomly generated bytes depending on the version, is also generated purely so the packet can be fully assembled and given a realistic size; it plays no functional role in the attack. Each packet is also generated with a payload (which plays no functional role in the attack). After packing the packet, a raw socket is opened and the packet is sent to the pre-known destination address, with `IP_HDRINCL` set, signaling to the kernel that it should be transmitted unmodified.

As such, the only changes that happen between different versions of the tool are only how the fragments are formed and scheduled. From the attacker's perspective, with each upgrade, the ultimate goal is to defeat the previous detector, whether by using overlapping fragments to go unnoticed, or by overwhelming it.

B. Detector

The main detector aim is to check incoming IPv4 fragments as they arrive and look for fragments that claim the same byte positions. It uses Scapy to read the packets and uses the source and destination IP addresses, protocol, and IP ID to decide which fragments belong to the same packet. The detector stores information about the fragments it receives and displays an alert when two fragments cover at least one of the same byte positions. To calculate each fragment's range correctly, it uses the IP total length minus the IP header length for the payload size. This excludes any Ethernet padding added after a short IP packet, which could otherwise make the fragment appear longer than it is, and produce false positives. The detector underwent three iterations: the first version checks only the most recent fragment, the second checks all fragments stored for that packet, and the third adds limits to prevent fragments from building up indefinitely. Each run writes its activity and alerts to a separate log file so the results can be reviewed afterward. From the detector's perspective, each upgrade aims to handle the new weakness exposed by the attacker's previous version, whether by flagging overlaps, accounting for out-of-order delivery, or enforcing limits on fragments' storage.

III. Tool Iterations

A. ***Version 1:*** *The Simple Overlap Fragments Attack*

1) Attacker: The first version of the attacker tool sends one fragmented IP packet, carrying a UDP payload, made up of a randomly chosen number of fragments (ranging from 3 to 6), all under the same randomly generated Identification Field. Among those fragments, all are considered normal (i.e. "well-formed") except for exactly one, randomly chosen, which is deliberately made to overlap (i.e. start inside the byte range of) the previous fragment before it, at a randomly chosen offset position. This overlapping fragment's size is also randomly generated and covers the remaining part of the previous fragment plus an additional non-overlapping part. The other normal fragments have a standard size of 32 bytes. When sending the packets, their order is not shifted, meaning that they are sent exactly in order, and the overlapping fragment is sent immediately after the fragment it overlaps, thus arriving one after the other. Notice that, on every run, the number of fragments and the overlap (including its size and position) are randomly chosen, such that no two runs are fingerprintable by the same fixed pattern. It is also important to notice that this is the simplest version of the overlapping fragment attack and mainly focuses on forming the overlapping fragment.

2) Detector: At the detector side, a received fragment goes through multiple checks before being stored.A packet with `MF = 0` and a fragment offset of zero is a normal, unfragmented packet, so the detector ignores it. Every other packet is treated as a fragment and tested for overlap. For this version, the detector is "naive": it only stores the previous fragment, and checks it against the current one, as it assumes that the incoming overlapping fragments are consecutive and arrive in order.

For each fragment, it calculates the start and end bytes, noting that the offset is measured in multiples of 8 bytes. It

then compares the fragment's range with previously received fragment from the same packet.

The start byte is the offset multiplied by eight, and the end byte is the start byte plus the payload length. To find the overlapping range, the detector takes the larger of the two start bytes and the smaller of the two end bytes.

If the overlap start is less than the overlap end, the fragments share at least one byte position, so the packet is flagged.

B. ***Version 2:*** *The Out-of-Order (O-of-O) Attack*

1) Attacker: The second version of the attacker tool, similar to the first one, sends one fragmented IP packet, all under the same randomly generated Identification Field. However, instead of having only one overlap, it has multiple ones (between 4 and 6, randomly chosen), constructed using the same Overlapping logic of Version 1, within a larger packet whose total number of fragments scales accordingly. What is different is that these overlapping fragments, when they are transmitted, are not sent immediately following one another (in a pair); instead, each pair is kept at least two fragment positions apart. This is done by inserting a separator between them, which is simply an "unrelated" normal (non-overlapping) fragment from elsewhere in the packet. Effectively, the fragments' offset, sizes, and payloads are identical to Version 1, with only the number of overlaps and the sequence in which they are sent changed.

2) Detector: The second attack exposes a weakness in that approach: another fragment can arrive between the two fragments that overlap, so storing only the previous fragment would not catch it. To handle this, Detector V2 stores all the fragment ranges it has seen for a packet (again identified by src, dst, IP ID, and protocol), and checks each new fragment against every stored range. It can therefore find an overlap even when the fragments arrive out of order. Lastly, when a fragment with MF=0 arrives, the detector sorts the stored fragments like a puzzle. Whenever the sequence is complete with no gaps, the detector assumes full packet transmission and correct reassembly, and flushes the entire fragments. Also, this detector ignores exact harmless retransmissions by comparing payload bytes. The downside is that a packet with many fragments takes more space to store and more time to check. This version of the detector displays insight into CPU usage and Average Processing Time per fragment.

C. ***Version 3:*** *The Overlapping Starvation Fragments Attack*

1) Attacker: The third version of the attacker tool deviates from the overlap-based evasion focus of the previous two and instead focuses on exhausting the resources of the detector (specifically, memory and CPU). Inspired by the FragmentSmack (CVE-2018-5391) vulnerability, it sends a larger user-configurable number of fragments (1,500 by default) under the same (randomly generated) Identification field, with random offset, size, and payload. Unlike the past versions, it never marks any fragment as the final one (i.e. the "more fragments" flag is never cleared). This effectively means that a detector which stores every fragment of an incomplete packet, in order to compare it against fragments received later, is forced to keep accumulating fragments indefinitely (i.e. it never receives the signal that the packet is complete and can therefore be released or finished comparing). Following this approach, using a large number of overlapping fragments and sending the fragments at a deliberately paced rate (10 ms between each), this attacker tool's version accumulates sustained resource usage over time, exhausting the detector's memory and processing capacity until it can significantly hinder its ability to operate (e.g. causing the detector to become unresponsive, crash, or stop detecting new fragments altogether).

2) Detector: Detector V3 keeps the overlap checks from V2 but adds a 30-second timeout and limits on what it can store: 4,096 datagrams, 128 fragments per datagram. These values are fully reconfigurable, and they're a design choice that depend on the application. The aim is to reduce the complexity of the detector. If an incomplete packet stays too long or reaches a limit, the detector removes its stored fragments and displays a warning. This lets it keep checking new traffic instead of allowing one incomplete packet to keep growing in memory.

IV. Testing And Results

The simulations were performed on two virtual machines (VMs), one Ubuntu machine and one Kali Linux, in VMware Workstation. The attacker is Kali, and the victim, running the detection tool, is Ubuntu. Host-only networking was used between the two VMs to keep both machines isolated from any external network.

The results below follow the three stages: *each attack version is first shown defeating the detector it targets, and each corresponding detector improvement is then shown catching it.*

Stage 1: Attacker V1 vs. Detector V1. The Version 1 attack was run against Detector V1 (*Figure* 1). As expected, because the overlapping fragment is transmitted immediately after the fragment it overlaps, Detector V1's single-slot comparison caught it on every run.

Stage 2: Attacker V2 vs. Detector V1, then vs. Detector V2. The Version 2 attack was first run against Detector V1. As shown in *Figure* 2, no alert was raised, confirming that separating each overlapping pair with an unrelated fragment defeats a detector that only compares consecutive fragments. The same attack was then run against Detector V2 (*Figure* 3), which correctly flagged the overlap regardless of arrival order (due to its full per-key fragment history feature).

Stage 3: Attacker V3 vs. Detector V2, then vs. Detector V3. The Version 3 attack was run against Detector V2 at

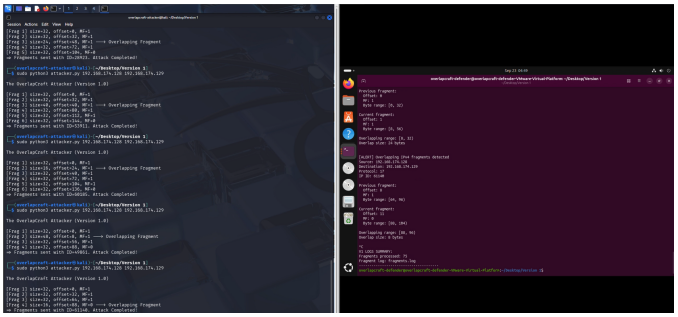


Fig. 1: Attacker V1 vs. Detector V1

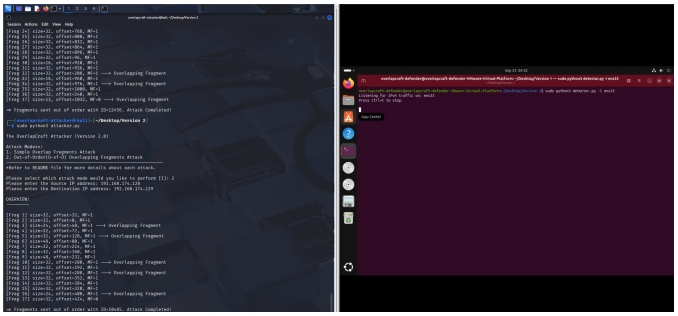


Fig. 2: Attacker V2 vs. Detector V1

increasing load. As the number of fragments sent scaled from 1,500 to 50,000, Detector V2's average processing time per fragment rose from 56.3 ms to 261.8 ms and its average CPU time per fragment rose from 20.9 ms to 89.0 ms, while every fragment remained stored indefinitely since no completing fragment was ever received, confirming the detector's unbounded per-datagram storage and comparison cost degrade it under sustained load (*Figure* 4). The same attack was then run against Detector V3, whose incomplete-datagram timeout and resource caps kept both metrics flat across the same load range, processing time from 15.7 ms to 15.2 ms and CPU time from 5.20 ms to 5.18 ms, and released stored fragments once the timeout expired (*Figure* 5). Detector V2's processing and CPU time both climb steadily as the fragment count increases, while Detector V3's remain flat regardless of load, demonstrating that the timeout and resource caps clearly neutralize the starvation attack's exhaustion effect.

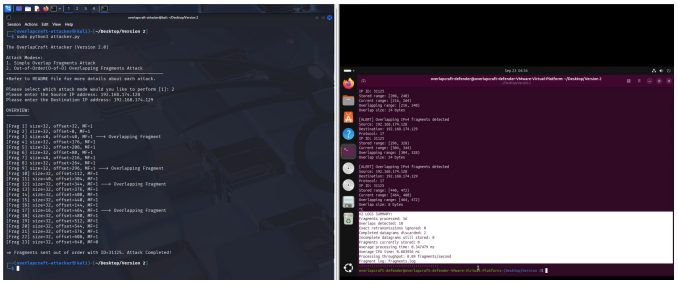


Fig. 3: Attacker V2 vs. Detector V2

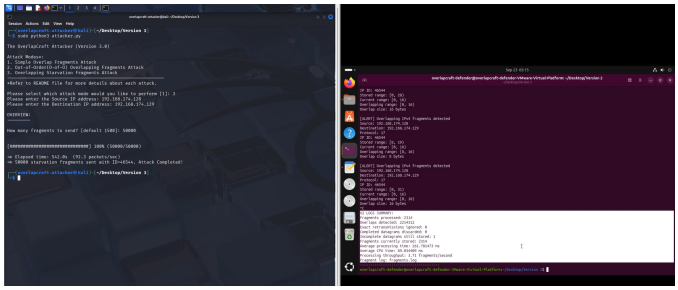


Fig. 4: Attacker V3 vs. Detector V2

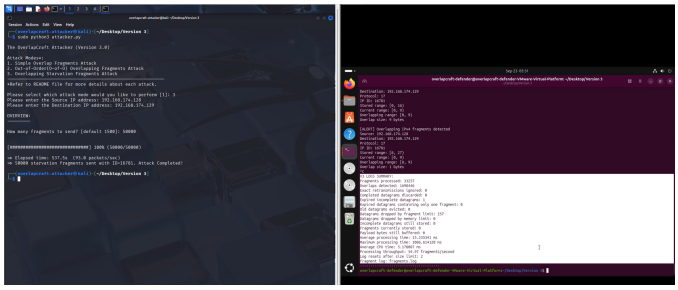


Fig. 5: Attacker V3 vs. Detector V3

V. Conclusion

We built three escalating rounds of overlapping-fragment attacks and matching detectors. Each attack broke the previous detector, whether by evasion or by exhausting its resources, and each detector was hardened to fix that exact gap. Results confirmed this: V2 evaded Detector V1 but was caught by V2, and V3 slowed Detector V2 under load while V3 stayed stable. This shows one protocol ambiguity can drive both evasion and denial-of-service attacks, and fixing one weakness doesn't guarantee protection against a different one.

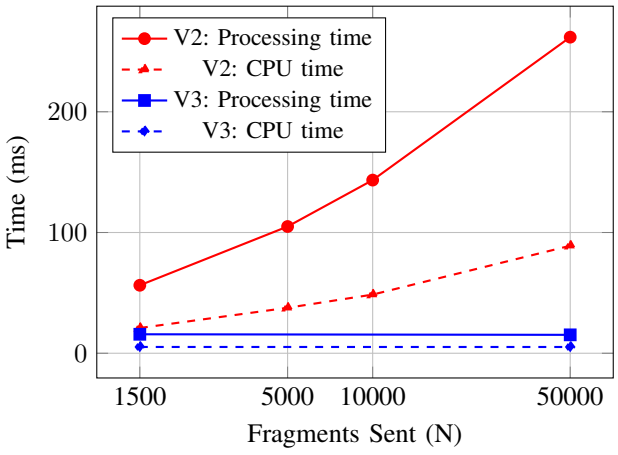


Fig. 6: Average Processing Time and CPU Time Per fragment under Attacker V3 vs. Detector V2, and vs. Detector V3

VI. References

Note on AI Assistance: AI was used only to fix grammar and phrasing; all technical content is our own.

Attacker References

1) RST Forums, "Raw Socket Programming in Python on Linux."
2) BrainKart, "Raw Sockets."
3) OneUptime, "IPv4 Packet Header Structure."
4) OneUptime, "Raw Sockets in Python for IPv4."
5) OneUptime, "Calculating the IPv4 Header Checksum."

Detector References

1) RFC 791, "Internet Protocol."
2) RFC 3128, "Protection Against a Variant of the Tiny Fragment Attack."
3) Scapy, "Sending and Receiving Packets."
4) Scapy, "Internet Protocol Layers."
5) EECE655L5, *Attacks and Countermeasures*, slides 18–22.
6) EECE655L8, *Prevention and Detection*, slides 16 and 24.

VII. Team Contributions

TABLE I: Team Contributions

Task	Contributor
Research and attack design (theoretical)	Both
Full attacker code implementation + debugging	Tamer Slim
Full detector code implementation + debugging	Zeinab Mazraani
Simulation + Recording	Tamer Slim
Video editing	Zeinab Mazraani
Report and supporting files	Both

VIII. Running the Simulations Locally

All codes are available on the project's GitHub repository, organized into separate folders for each version.

- `cd` into the folder of the version you want to run (`Version 1`, `Version 2`, or `Version 3`), then run the scripts from there.
- Attacker V1 takes the source and destination IPs as command-line arguments: `sudo python3 attacker.py <srcIP> <dstIP>`.
- Attacker V2 and V3 are interactive: run `sudo python3 attacker.py` and follow the prompts to select an attack mode and enter the IPs.
- Detector (any version): run `sudo python3 detector.py -i <interface>` for live capture, or `-r <pcap_file>` to analyze a saved capture.